# 12. Simulating the LM317 Controller (and other circuits)



## Table of Contents



### Objective

Model our Arduino controlled voltage regulator circuit, and compare the simulation to PWM calibration data saved in the previous chapter. I'll also demonstrate two other circuit simulations: a multistage audio amplifier/mixer/filter, and a 60Hz filter. This should teach MATLAB skills and reinforce how and why circuits work.

### constrain.m – a helper function

This function, saved in your PATH, will save a couple lines of code repeatedly in each mode. It's modeled on Arduino's constrain function (https://docs.arduino.cc/language-reference/en/functions/math/constrain/) that "clips" Vmin ≤ Vout ≤ Vmax.

```
function [xout] = constrain(xin, xmin, xmax)
% function [xout] = constrain(xin, xmin, xmax)
%   arduino's constrain function, xmin<=xout<=xmax
xout = xin;
xout(xout < xmin) = xmin;
xout(xout > xmax) = xmax;
end

% testing: a = -5:5, b = constrain(a,-1,3)
```

Logical indexing (e.g., `xout(xout < xmin) = xmin`) is less verbose, and probably more efficient, than `if/else` constructs. `(xout < xmin)` is a vector of Boolean (true/false) values. The corresponding true elements of `xout` are set equal to `xmin`, without explicit looping over the vector elements.

## Modeling the LM317 controller

A good approach to modeling anything is start simple. Make assumptions that simplify the model, even if they're a little wrong. If you're lucky, the simple model captures the main aspects and it may not be necessary to capture more details. Because of inherent complexity, your first model may be horrendously wrong, due to all sorts of bugs – typographic errors, inconsistent units, and more fundamental logical bugs. These are easier to find and fix in simple models. Once debugged, you're in a better place to add additional complexity including higher order terms and corrections. You also can compare models to quantify these effects. In a metaphor, add the spices one at a time to the soup, and taste it before adding the next one. You might be satisfied and call it done without having to dice and saute the onions ...

### Parameters and measurements

Start by defining and measuring key parameters. Just the first ones are used in the simplest model, but additional parameters were added to the bottom of the list for subsequent models, thereby keeping most parameters in one code block. Substitute your measured values. Always remember to measure resistances with the circuit unpowered, including ensuring no power comes from active PWM pins or other sources.

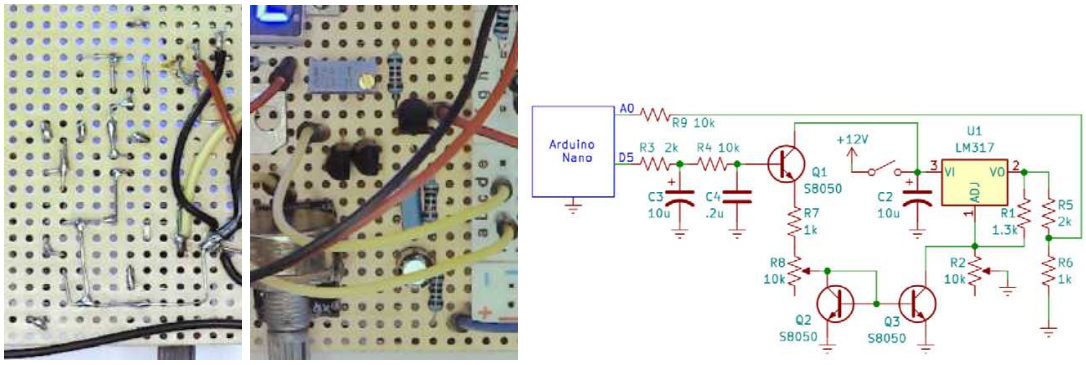



Figure 1. LM317 controller photos of back and front of board and schematic.

Which voltages or pwm values should we model? The same ones we measured.

```
%% parameters -- measure all R's with power OFF
pwm = (50:5:255)'; % PWM values (at pin D5) to simulate
vref = 4.752; % volts on vref pin
vref317 = 1.252; % measured with R2 at minimum
Vmax = 11.12; % measured maximum voltage out
R1 = 1215; % ohms measured between LM317 pins 1&2 (power off)
R2 = 9.57e3; % ohms measured at maximum
% NOTE: you can measure R2 between the center and UNCONNECTED
% pin with knob at MINIMUM -- i.e., measuring the other side
R7 = 3448; % ohms = R7 + R8, measured from Q1 emitter to Q2
% or Q3 base (middle pin)
Vbe = 0.6; % estimated transistor B-E voltage when turned on
```

Load your calibration measurements to compare with the model. You should have saved vectors `pwm` and `v317` in a file called `pwmcaldata.mat` in the previous chapter. This may make finding the file easier for you, the `if` statement loads the file just the first time the script runs.

```
if ~exist('v317','var')
    try
        load pwmcaldata.mat
    catch
        disp('find file: pwmcaldata.mat')
        [fname, path] = uigetfile('.mat',"find pwmcaldata");
        load([path fname]);
        fprintf('loaded measurements from: %s%s\n', path, fname)
        clear path fname
    end
end
```

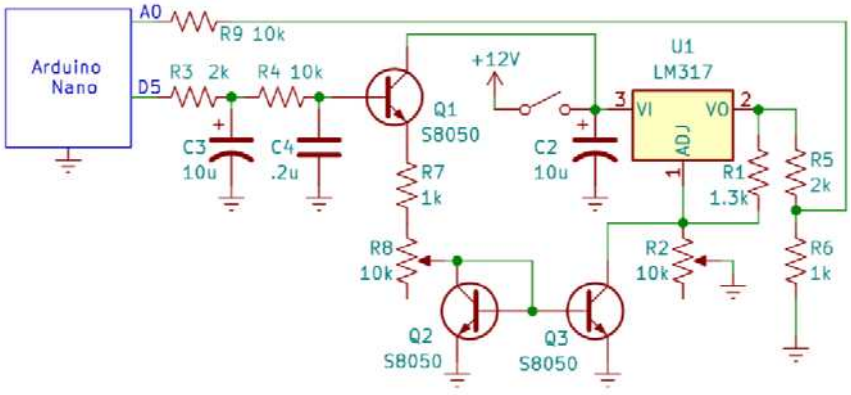
There are many ways to describe a model, but let's go right into MATLAB code and some comments (vs. e.g., flowcharts, pseudocode, UML, etc.) I generally try to write "self-documenting" code and minimize redundant comments. Too many times I've come across inconsistent code and comments, like this simplified line:

```
a = b * 3; % a is twice b
```

If you read the executable part, you'd know exactly what it does. But the comment causes you to freeze. Where is the bug? Should the 3 be a 2? Perhaps an earlier version (when the comment was written) executed a = b * 2, but the code evolved and the comment didn't. In a Darwinian sense, evolutionary pressure drives code towards improvement, but comments aren't tested, bugs in comments are "neutral", and they tend to persist or grow. Erroneous comments are more disruptive than no comments, so I recommend minimizing comments, keeping them general vs. specific (e.g., in the example above, `% a is rescaled b` might remain valid for longer), but best is to write self-evident or self-documenting executable code. What do you think?

### Model 1: Basic

Follow the signal from D5 in the schematic as you read the rest of this paragraph and the code below. The low pass filter turns the pwm value into a dc voltage at Q1's base, as stated in the first two lines of code. Q1's emitter voltage is Vbe less than its base voltage (when Q1 is active). Q2 (and Q3) emitter are at V=0, so their base voltages are Vbe ~ 0.6V (when they are active). Q2's emitter is connected to its base, hence also Vbe. Now we have the voltages on both sides of the emitter follower and mirror's resistor (R7+part of R8), Ohm's Law gives `iMirror`. I hope you find it easier/clearer to just read the 15 lines of code and comments below that constitute the entire model (including displaying it).



```
vpwm = pwm * vref / 255; % DC voltage on pin D5 AND base of Q1
Q1baseVoltage = vpwm; % the PWM filter's gain = 1 at DC
Q1emitterVoltage = Q1baseVoltage - Vbe;
% when Q2 is active, its base Vbe~constant, and Vce=Vbe
Q2collectorVoltage = Vbe; % because its connected to base
iMirror = (Q1emitterVoltage-Q2collectorVoltage) / R7; % Amps
% if Q1 isn't conducting then iMirror = 0:
iMirror(Q1baseVoltage < Vbe + Q2collectorVoltage) = 0;
iR1 = vref317 / R1; % Amps, from LM317 datasheet
iR2 = iR1 - iMirror; % iR1 = iR2 + iMirror, by KCL
VR2 = iR2 * R2;
DCVout = VR2 + vref317;
DCVout1 = constrain(DCVout, vref317, Vmax);
figure; plot(pwm, [v317 DCVout1]);
xlabel('PWM value'); ylabel('DCVout');
dots; grid; axis tight; legend('measured', 'model 1')
```

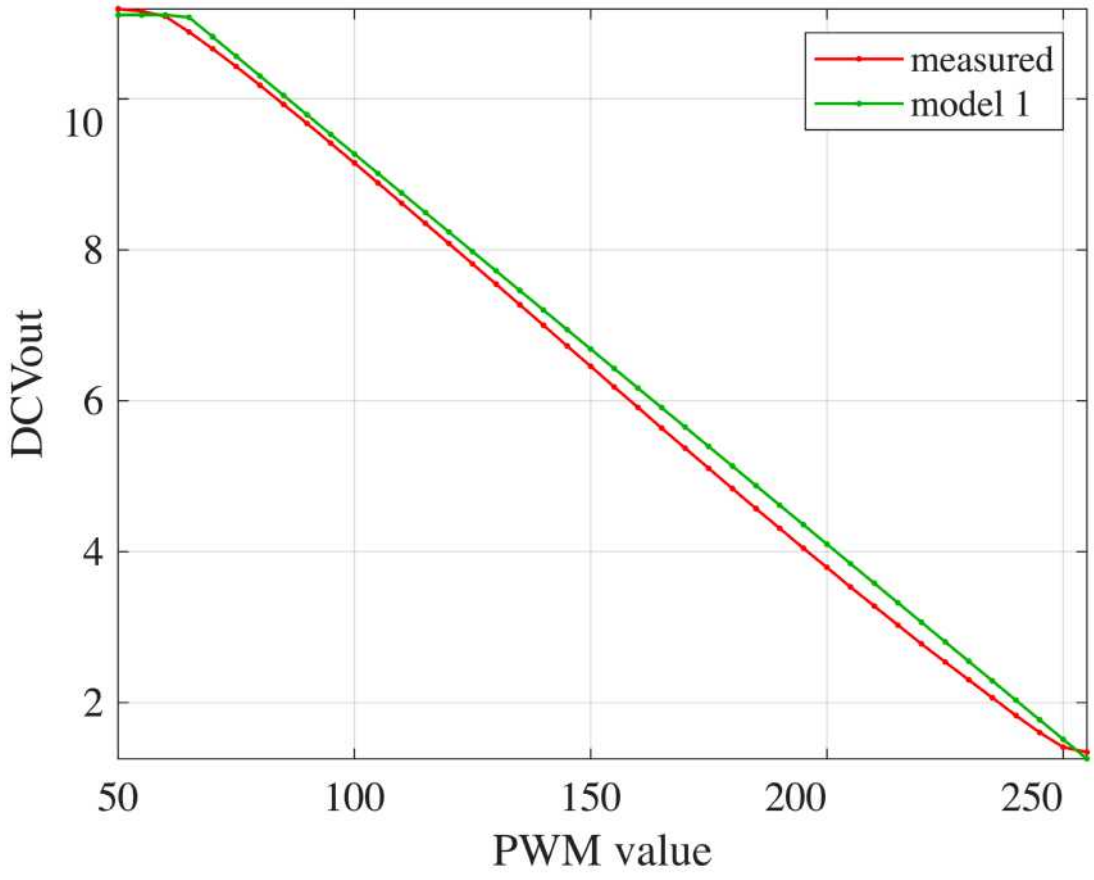



Figure 2. Model 1 vs. measurements.

Reasonable (?) agreement with experiment. This is a very different modeling approach than conventional circuit simulation programs like SPICE, Eagle, KICAD, etc., where most component parameters and rules are "under the hood". They (sometimes) give accurate results, but offer less intuition about how and why the circuit works than our dozen code lines above.

This model is yet another tool to explore and test the circuit. E.g., want to see the effects of changing a resistor's value, then change the corresponding parameter and run the code again ... With several quick "drylab" iterations, you may improve the fit between model and experiment, which may point you towards rechecking corresponding measured parameter values.

Should we expect this model to exactly fit the data? Definitely not. Richard Feynman famously said, paraphrasing, "A theory should not agree with all the measurements, because not all the measurements are correct". In this case we *know* this model is wrong. Transistors don't instantly turn on when $V_{BE} = 0.6V$. LM317's $I_{ADJ} \neq 0$, as modeled. Q1's base current causes an $iR$ voltage drop across the PWM filter resistors, etc. Importantly, the model captured the circuit's main characteristic behaviors without adding all those extra seasonings to the soup. The simpler model independently confirms that we didn't somehow get lucky with the circuit (as rare as good luck is). Let's quantitatively explore some of the secondary factors just mentioned.

### Model 2: Adding Q1 beta

$1/\beta$ of Q1's emitter current, `iMirror`, comes from its base, sourced from Arduino pin D5, through R3, and R4 of the PWM filter. Fig. 3 shows simplified equivalent circuits relating D5 voltage to `iMirror`. The PWM filter capacitors' impedance to ground are infinite at DC so they're not shown. (Of course, without the filter capacitors the voltage on D5 would be switching between 0 and Vref ~970 times per second, Fig. 2 of Chapter 10. The capacitors play an important role in reality, that we capture by modeling D5 as an analog dc voltage.) When Q1 and Q2 are active, their two $V_{BE}$ junctions are equivalent (to first order) to 0.6V batteries. And the current gain of Q1 can be modeled by simply multiplying $R_E$ by $\beta$ (why emitter followers are sometimes called "transimpedance amplifiers"). You might be tempted to use the relationship, $I_B =$ `iMirror` $/ \beta$, but the `iMirror` we computed ignored the loading effect we're now trying to model now, so it's also unknown. Combining the two $V_{BE}$ since they are in series, and all the series resistances lets us model Q1's $I_B$ by the 3 equivalent circuits in Fig. 3.

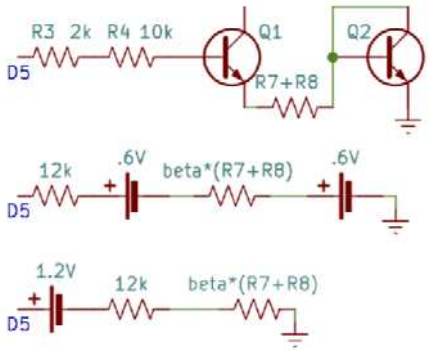



Figure 3. Equivalent, simplified DC current paths from D5 to ground.

```
R3 = 12e3; % sum of R3 and R4
beta = 200; % guesstimated transistor gain
Q1baseCurrent = (vpwm - 2*Vbe) / (R3 + beta * R7);
Q1baseCurrent(Q1baseCurrent < 0) = 0; % when vpwm < 2*Vbe
Q1baseVoltage = vpwm - Q1baseCurrent * R3;
% the rest of the model stays the same
Q1emitterVoltage = Q1baseVoltage - Vbe;
Q2collectorVoltage = Vbe;
iMirror = (Q1emitterVoltage-Q2collectorVoltage) / R7; % A
% if Q1 isn't conducting then iMirror = 0:
iMirror(Q1baseVoltage < Vbe + Q2collectorVoltage) = 0;
```

```
iR1 = vref317 / R1; % Amps, from LM317 datasheet.
iR2 = iR1 - iMirror; % iR1 = iR2 + iMirror, by KCL
VR2 = iR2 * R2;
DCVout = VR2 + vref317;
DCVout2 = constrain(DCVout, vref317, Vmax);
plot(pwm, [v317 DCVout1 DCVout2]);
xlabel('PWM value'); ylabel('DCVout'); grid; axis tight;
legend('measured', 'model 1','model 2')
```

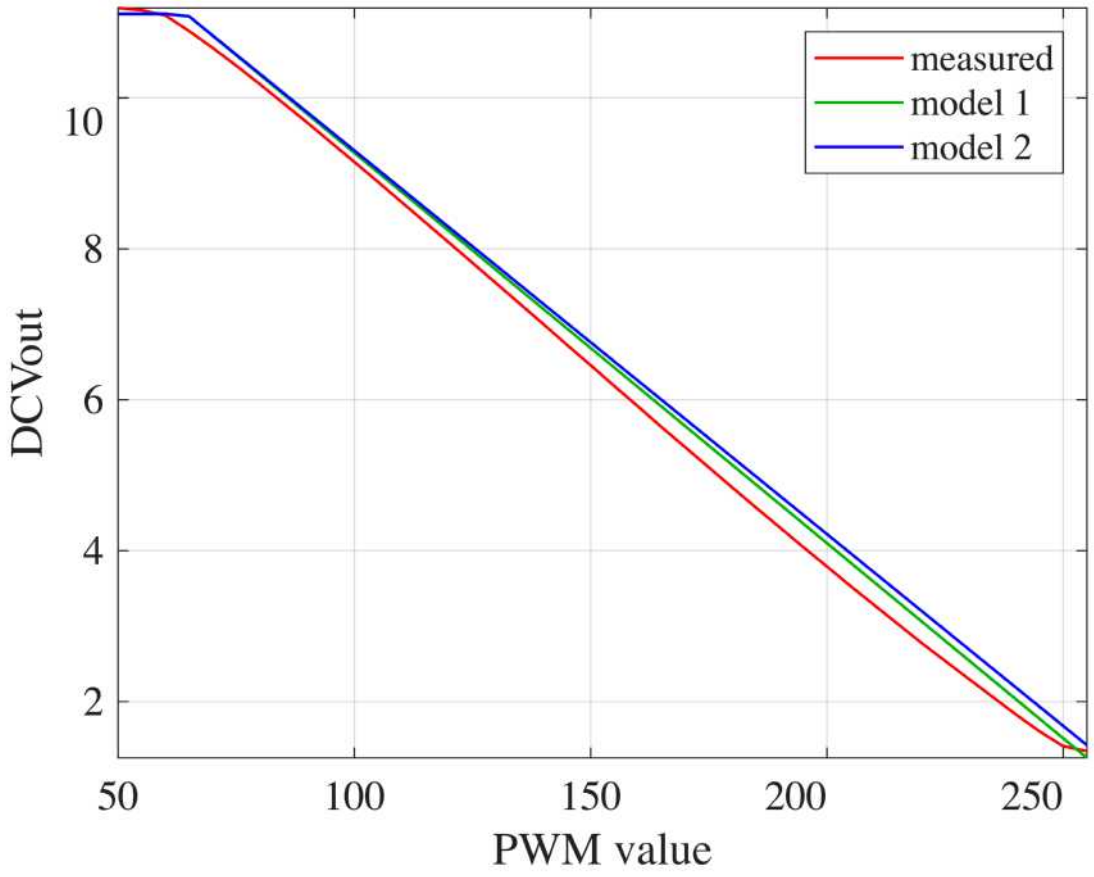



Figure 4. Modeling Q1's finite beta has a small effect at increasing PWM values.

**Model 3: Adding i_ADJ current**

We've thus far ignored current from LM317's ADJ pin, so let's add this parameter. The datasheet says this can be up to $50\mu A$, but is typically <~$20\mu A$. The code below recycles parameters from Model 1 and 2 that aren't affected by adding $I_{ADJ}$ to iR2. Experiment with different (plausible) values.

```
iAdj = 20e-6; % LM317 ADJ pin current est. from datasheet
iR2 = iR1 - iMirror + iAdj; % iR1 = iR2 + iMirror + iAdj, by KCL
VR2 = iR2 * R2;
DCVout = VR2 + vref317;
DCVout3 = constrain(DCVout, vref317, Vmax);
```

```
plot(pwm, [v317 DCVout1 DCVout2 DCVout3]);
xlabel('PWM value'); ylabel('DCVout'); grid; axis tight;
legend('measured', 'model 1','model 2','model 3')
```

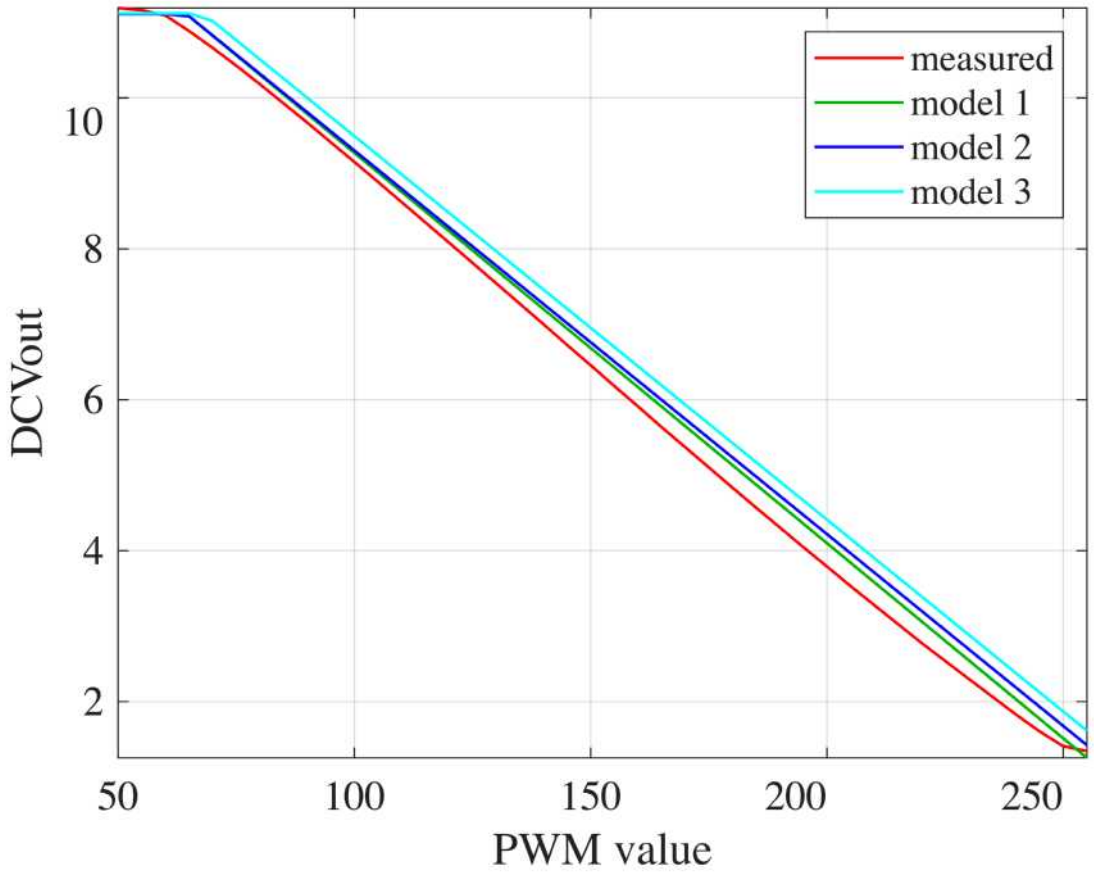



Figure 5. iAdj moves the curve for all PWM values.

**Model 4: Mirror beta mismatch**

So far the correction terms are relatively small, confirming the decision to ignore them in the first place – although now we know quantitatively *how* they affect the model. However, these effects move the model results *further* from the measurements suggesting there's a more important factor we still missing.

In Chapter 7 I measured 3 transistors and found $\beta$ = [208.1 276.4 212.0] – widely variable – `std(beta) = 38`, Coefficient of Variance (CV = `std(beta)/mean(beta)` = 16.5%. Repeating measurements on the same transistor gave the same result with CV < 0.005 = 0.5%. Thus it's likely Q2 and Q3 have different $\beta$'s. How would that affect the circuit? (BTW, this is a reason discrete transistor current mirrors are uncommon, although they are common building blocks on integrated circuits where $\beta$ is much more consistent.)

Q2 and Q3 have the same $V_B$ and $V_E$ in the circuit, which suggests they have the same $i_B$. So each transistor's $i_{C[1,2]} = \beta_{[1,2]} i_{B[1,2]}$, and $i_{C1}/i_{C2} = \beta_1/\beta_2$. Adjusting pot $R_8$ sets $i_{C1} = (V_{ref} - 2V_{BE})/R_8$, so $i_{C2} = (V_{PWM} - 2V_{BE})\beta_2/(\beta_1 R_8) = k_1 V_{PWM} + k_2$ where slope $k_1 = R_8\beta_2/\beta_1$is determined by adjusting the value of $R_8$, accounting for any (constant) mismatch of the transistor's $\beta$'s. In the model the beta ratio can

be encoded with a "fudge factor" = $\beta_{Q3}/\beta_{Q2}$ that multiplies `iMirror`, empirically tweaked to fit the measurements. I tried, fudge = 0.9, then 1.1, and left it at:

```
fudge = 1.05; % current mirror mismatch
iR2 = iR1 - iMirror * fudge; % iR1 = iR2 + iMirror, by KCL
VR2 = iR2 * R2;
DCVout = VR2 + vref317;
DCVout4 = constrain(DCVout, vref317, Vmax);
plot(pwm, [v317 DCVout1 DCVout2 DCVout3 DCVout4]);
xlabel('PWM value'); ylabel('DCVout'); grid; axis tight;
legend('measured', 'model 1','model 2','model 3', 'model 4')
```

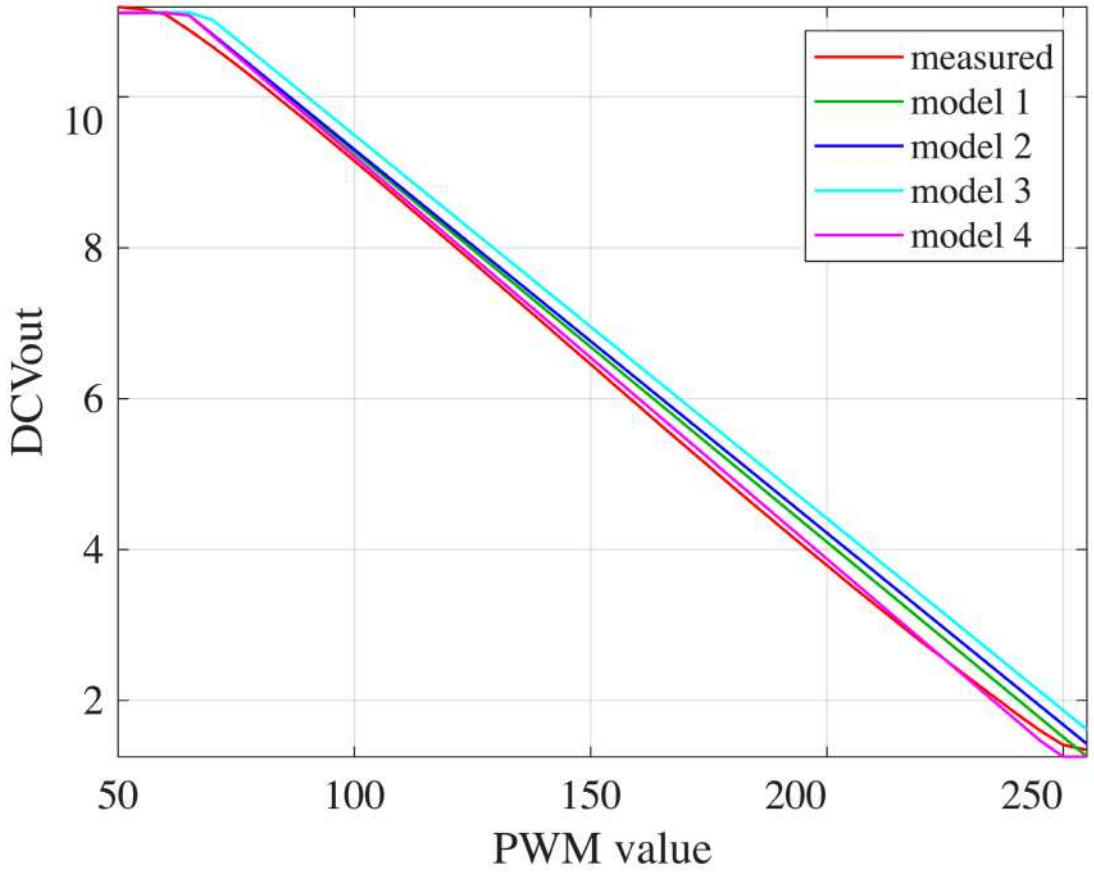



Figure 6. Empirically picking a fudge factor (= $\beta_{Q3}/\beta_{Q2} \neq 1$) achieved better quantitative agreement between model and measurements.

### Difference plots

Plotting the differences between models and measurements is somewhat akin to rotating Fig. 6 so its slope is horizontal. I also flipped the x-axis to `vout` – what we really think about. The first 3 lines keep the colors consistent with the previous figures.

```
clf;  % new figure window
set(gca,'ColorOrderIndex',2) % start with the 2nd color
hold on; % necessary to hold ColorOrderIndex
plot(v317, [DCVout1 DCVout2 DCVout3 DCVout4] - v317);
xlabel('output voltage'); ylabel('model - measurements / volts');
dots; grid; axis tight;
legend('model 1','model 2','model 3','model 4','location','best')
```

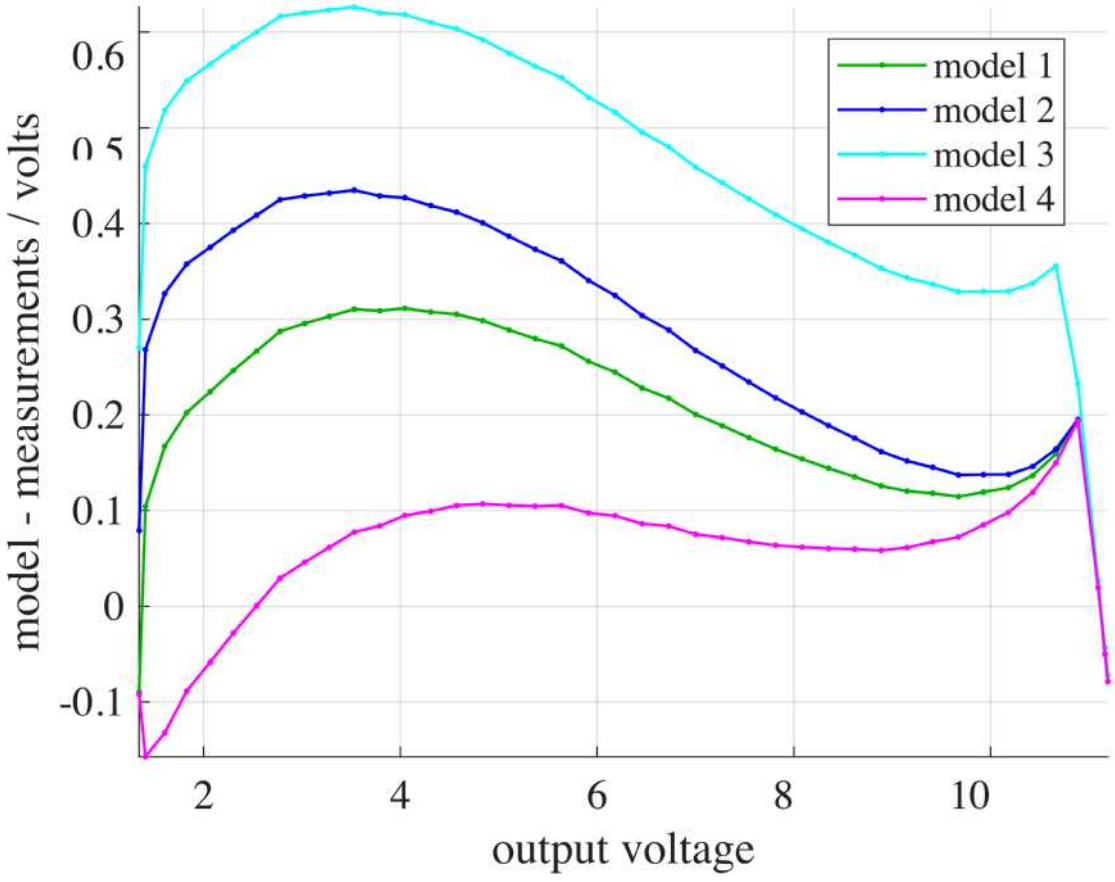



Figure 7. Error vs. output voltage for the 4 models.

```
rms([DCVout1 DCVout2 DCVout3 DCVout4] - v317)
```

```
ans = 1x4
    0.2145    0.2996    0.4739    0.0897
```

Is it unethical to fudge data and models to fit each other? Not if we are both honest and transparent about what the goals are, which in this case is to better understand how or why our circuit works. The models suggests that our two transistors have mismatched betas of order `fudge` (which is quite plausible), that `iAdj` *is* relatively insignificant, as is the emitter follower's loading of the PWM low-pass filter. Was it worth the effort to determine and quantify these things? What else did you learn?

## Modeling a multi-stage audio intercom system

Next consider this first-principles model of an audio intercom system for an access control/security system. Inside and outside a remote gate are two electret microphones (and speakers, not shown) and a buzzer – like a door bell. The first op-amp (left) amplifies and filters the microphone signal. The second amp combines ("mixes") the buzzer and microphone signals. The third stage mixes the two intercom boxes (inside and outside the gate) with audio output from a Raspberry pi computer, that plays jingles indicating when the gate is left open and announces other events by injecting sounds. Every input has an RC high pass filter, and every feedback has an RC low pass filter, that together make 4 stages of bandpass filter with steep transitions. The code below models the gain of each stage and the system gain, and is plotted along with manual measurements made with a function generator providing sine wave input to C8 and a (wide bandwidth) AC voltmeter measured output at V4.

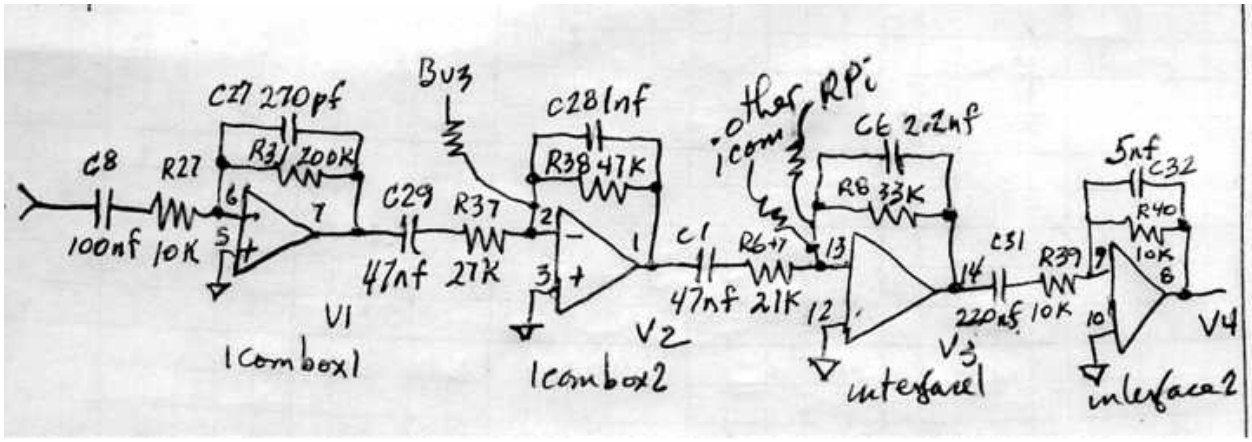



Figure 8. 4 stage audio amplifier/mixer/filter for an intercom system.

The model uses the `zc` (Chapter 9) and `zp` (Chapter 10) functions that compute impedances of capacitors and parallel impedances, to compute gains at all frequencies for each op-amp stage in one line. Recall, each inverting op-amp stage has gain $= \frac{Z_{FEEDBACK}}{Z_{INPUT}}$, and the numerator and denominator are vectors of complex numbers representing impedances at each modeled frequency. The first line clears the `hold on` from the previous difference plot. Next line specifies the frequencies to be modeled. The next two lines define the stage's parameter values, and in one line the stage's gain is computed (for all frequencies – note the explicit pointwise division and multiplication, `./` and `.*`). The last few lines superimpose empirical measurements of the circuit.

```
figure;
freqs = logspace(1,4,200)'; % frequencies
%% first stage of microphone amp
r31 = 200e3; c27 = 270e-12;
r27=10e3; c8 = 100e-9;
gain1 = zp(r31, zc(c27,freqs)) ./ (r27+zc(c8,freqs)); % = zfeedback/zinput
%% 2nd stage of microphone amp
c29 = 50e-9; r37 = 27e3;
r38 = 47e3; c28 = 1e-9;
gain2 = zp(r38,zc(c28)) ./ (r37+zc(c29));
%% interface 1
r6 = 21e3; c1 = 50e-9;
r8 = 33e3; c6 = 2.2e-9;
gain3 = zp(r8, zc(c6)) ./ (r6 + zc(c1));
%% interface 2
r39 = 10e3; c31 = 220e-9;
```

```
r40 = 10e3; c32 = 5e-9;
gain4 = zp(r40, zc(c32)) ./ (r39+zc(c31));
%% 4 stages together
gout = gain1 .* gain2 .* gain3 .* gain4;
loglog(freqs, abs([gain1 gain2 gain3 gain4 gout]))
grid; xlabel('Hz'); ylabel('gain');
%% add manual measurements
hold on;
vin = 10; % mVrms measured at mic input
freq = [50 100:100:1000 1200:200:2000]';
von14rms = [11 72 248 360 413 438 447 447 441 432 420 391 359 326 295 265 ]';
freq2 = [2500 3000:1000:10000]';
von14rms2 = [201 152 89 54 35 24 17 13 10]';
loglog([freq; freq2], [von14rms; von14rms2]/vin,'d-k')
legend('stage1','stage2','stage3','stage4','product', 'measured','location','best')
hold off;
```

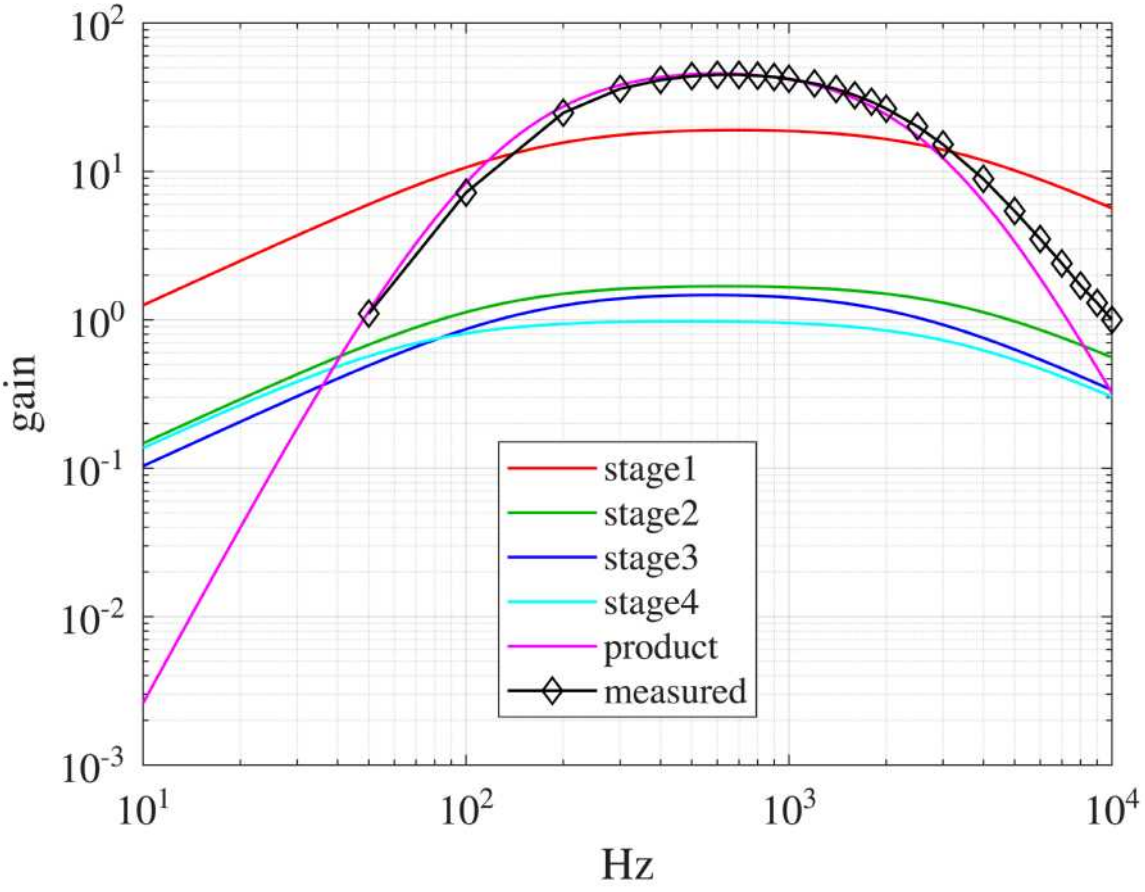
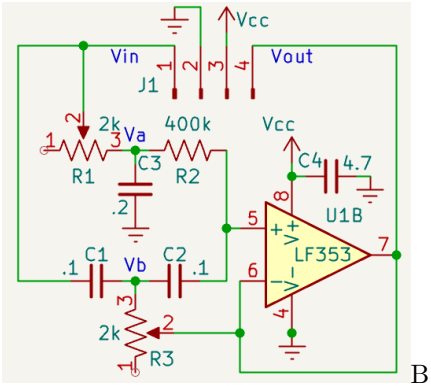



Figure 9. Frequency domain simulation and measurements of the circuit in Fig. 8.

The measurements suggest a low pass filter corner is too high (right) and directed inquiry revealing C31 was erroneously 50nf instead of 220nf. The system achieves 50% attenuation at 172Hz & 2-2.3kHz and 99% attenuation at ~40Hz and ~20kHz, reducing hiss & airplane noise and improves hearing voices over truck engines, footsteps over birds, etc.

## Modeling a "Twin-T" 60Hz notch filter

60 Hz is a common form of electronic noise in the Western Hemisphere radiating from our electric power system (50Hz in Europe). This Twin-T notch filter is a highly frequency-selective circuit that attenuates 60Hz signals, yet passes both higher and lower frequencies (https://en.wikipedia.org/wiki/RC_oscillator#Twin-T_oscillator hints at the theory of operation). Let's model it from first principles.

A. 

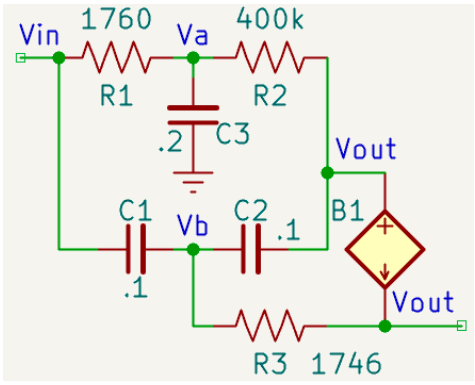
B. 


Figure 10. A. Schematic of a Twin-T filter module, drawn this way to highlight the twin "T"s, (R1,R2, C3 and complementary C1, C2, R3), on the positive input of an op-amp follower. B. Redrawn to facilitate applying KVL and KCL for nodal or loop analysis

The op-amp follower loads the filter with a nearly infinite impedance, drawing zero current on pin 5, while providing a small Thevenin impedance in series with the output $V_{OUT}$. Hence the op-amp can be replaced conceptually with the current source B1 in Fig. 10B, to remind us that although the voltages are the same on pins 5, 6, and 7 (by the op-amp's "golden rules", see Chapter 8), the op-amp may source/sink unknown currents from the power supply (Vcc on pin 8 and ground on pin 4) to achieve the same voltages on its two inputs (and output in this case of a follower). There are 4 unknown voltages, Vin, Va, Vb, and Vout, so we need 4 independent equations. Without loss of generality, let Vin = 1 (we want the gain = Vout/Vin = Vout in this case). Sum the currents entering each node (Kirchhoff's Current Law, KCL):

1) (Vin-Va)/R1 + (V5(=Vout)-Va)/R2 + (0-Va)/ZC3 = 0 % KCL at Va

2) (Vin-Vb)/ZC1 + (Vout-Vb)/ZC2 + (Vout-Vb)/R3 = 0; % KCL at Vb

3) (Va-Vout)/R2 +(Vb-Vout)/ZC2 = 0; % no current flows into or out of the + input, op-amp "golden rules"

Note that R1 and R3's values are what's between their pins 2-3, not the ~2k value between pins 1 & 3. Let's write those 3 equations in matrix format, [[A]][x] = [b], where [x] = [Va; Vb; Vout]. I had to write it out on paper (after first trying a few times going directly into code, and missing terms and signs and getting wrong answers):

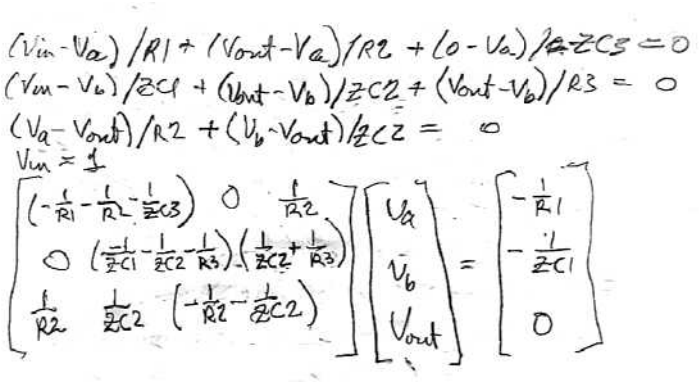


Figure 11. KCL equations in linear and matrix forms.

Now tell MATLAB to solve the matrix equation at a set of frequencies. Rather than adding a 3rd dimension to the matrix problem for frequency, I opted to loop over each frequency:

```
R1 = 1760; R3 = 1746; % measured values (Ohms)
R2 = 400e3;
C1 = 0.1e-6; C2 = 0.1e-6; C3 = 0.2e-6;

% start with a wide freq range so you don't miss the notch if parameters are off
% fq = logspace(1,3,2000)';
fq = linspace(50,70,1000)'; % then narrow the f-span around the notch
ZC1 = zc(C1, fq); ZC2 = zc(C2); ZC3 = zc(C3);
Gain = zeros(size(fq));

for k = 1:length(fq) % each frequency
    A = [(-1/R1 -1/R2 - 1/ZC3(k)) 0 1/R2; ...
        0 (-1/ZC1(k) - 1/ZC2(k)-1/R3) (1/ZC2(k)+1/R3); ...
        1/R2 1/ZC2(k) (-1/R2-1/ZC2(k))];
    b = [-1/R1; -1/ZC1(k); 0];
    x = A \ b; % invert or solve the matrix equation
    Gain(k) = x(3); % Vout is the 3rd component
end

figure;
loglog(fq, abs(Gain)); xlabel('freq / Hz'); ylabel('gain'); grid;
```

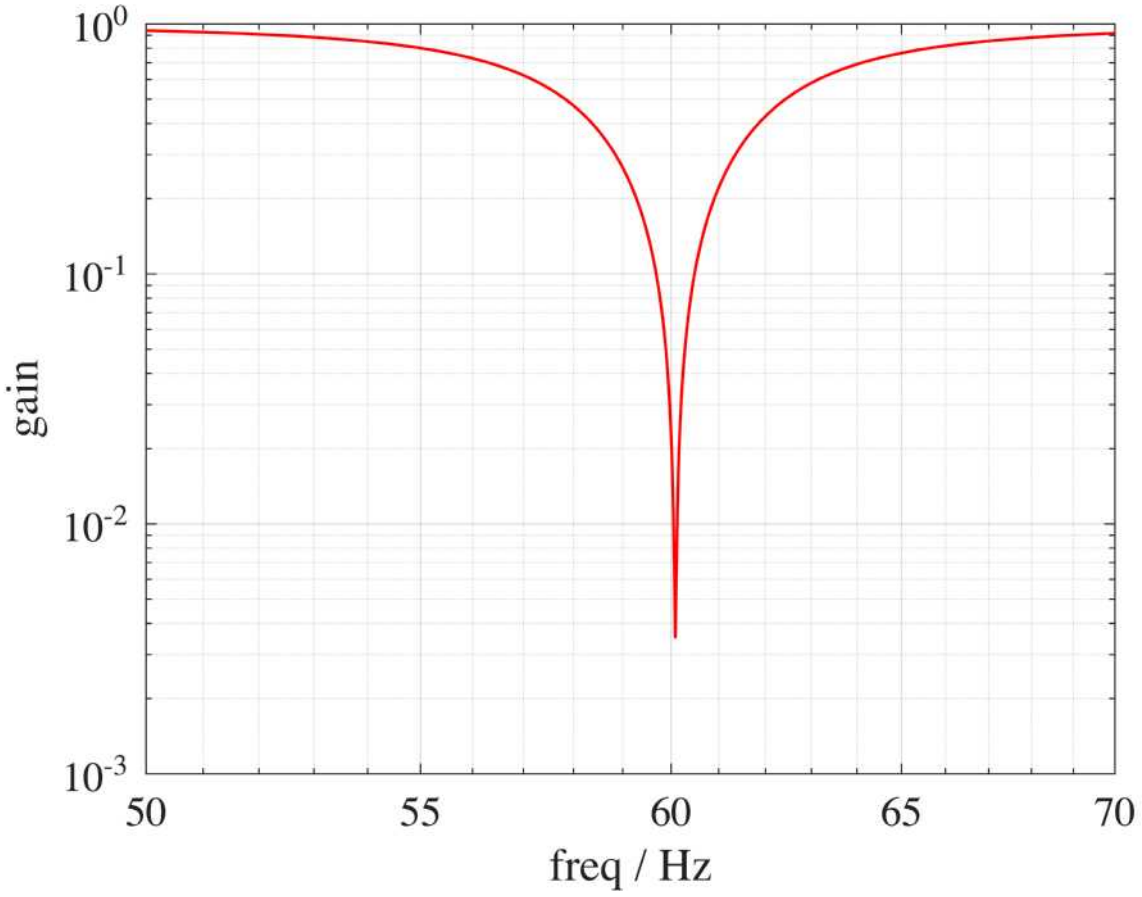



Figure 12. Simulated Twin-T notch filter gain.

```
% Find notch
[gmin, indx] = min(abs(Gain));
fprintf('Notch depth = %.3e (linear) = %.2f dB at %.2f Hz\n', gmin, 20*log10(gmin), fq(indx));
```

```
Notch depth = 3.525e-03 (linear) = -49.06 dB at 60.09 Hz
```

If you measured gain at intervals of 5 or 10Hz, you'd have no clue how this circuit behaves in between. Resonant circuits like this can be very non-linear, and require dense sampling to capture their behaviors. Slight variations in component values (e.g., capacitors can change with temperature and humidity) can have a large effects. This filter demands high quality film capacitors, not ceramic or electrolytic dielectrics. The simulation with the provided component values has $f_0 \sim 60.09$Hz, with ~10x more attenuation than at the target 60.00Hz, so tweaking parameters is warranted.

### Quality Factor, Q

See https://en.wikipedia.org/wiki/Q_factor and Chapter `s9_Resonance` for background.

**Definitions:** $Q \equiv \frac{f_r}{\Delta f}$, where $f_r$ is the resonant frequency, and $\Delta f$is the **resonance width** or full width at half maximum (FWHM). We just printed the numerator (`fq(indx)`). $\Delta f$is the width of the curve where

the *power* decreases by 1/2. And since $P = V^2/R$, $\Delta f$one might think is the width of Fig. 12 where it intersects `gain =` $\sqrt{1/2}$. Maybe before *trying* to solve that, think of at least 2 or 3 different ways you *might* solve it. Then pick the easiest one and try it before continuing.

**A Solution**

- Did you think of using Tool Tips on Fig. 12?

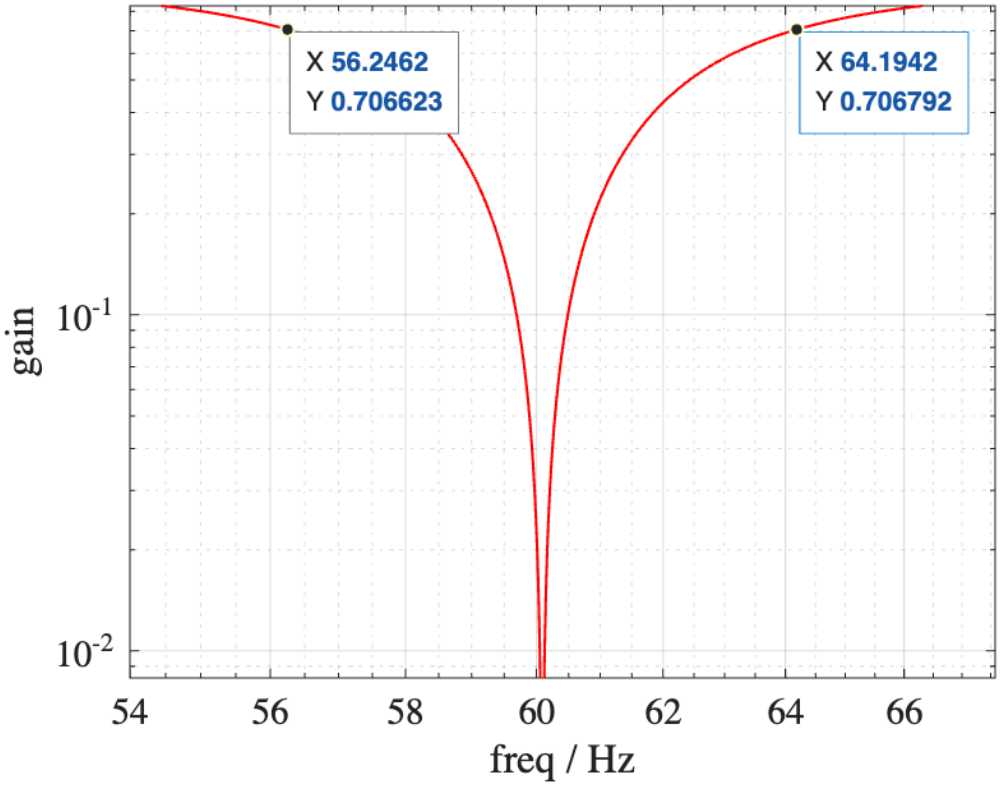



Figure 13. Approximate intercepts where the gain ~ $\frac{1}{\sqrt{2}}$

I zoomed in first (and zoomed too much – I expected to be measuring much lower into the notch!). I think these are the nearest data to y = $1/\sqrt{(2)}$ = .7071... Thus $\Delta f \approx$ 65.19-56.25 = 8.9, and Q = 60 / 8.9 = 6.7. And that's a wrong answer for a notch filter, because ...

For a notch filter, $\Delta f$should be measured as the width where the power *increases* by a factor of 2 from its *minimum* value (which we just computed):

```
fprintf('y intercept for measuring Q = %.2e\n', gmin * sqrt(2))
```

```
y intercept for measuring Q = 4.99e-03
```

We really need to zoom in now, but that reveals that the data points are too far apart. Try another method, e.g., print the numbers instead of the graph:

```
fprintf('\t Hz       Gain'); % header
```

```
         Hz       Gain
```

```
[fq(indx-2:indx+2) abs(Gain(indx-2:indx+2))]
```

```
ans = 5x2
   60.0501    0.0106
   60.0701    0.0061
   60.0901    0.0035
   60.1101    0.0062
   60.1301    0.0107
```

The Gain = 0.005 intercept are closer than the adjacent points. Best estimate is $\Delta f <$ 60.11-60.07 = 0.04Hz. So Q > 60 / .04 = 1500. That's not an accurate estimate (it's not even an estimate, but a one-sided limit), but sometimes that's adequate information to make decisions. If a more accurate estimate is warranted, how to proceed? Again I encourage you to think of at least 2-3 potential solutions before attempting any.

The simplest solution I think is replacing one line of code above (copy and comment out the original line for easy undo).

```
% fq = linspace(50,70,1000)'; % then narrow the f-span around the notch
fq = linspace(fq(indx-1),fq(indx+1),1000)'; % why a different size?
```

This just zooms in the frequency axis, computing the same number of data points, now in the $f_r \pm .02$Hz range of interest. Then rerun the file, copy/save the results, and revert to the original. Here are the results with my original parameters (R1 = 1760; R3 = 1746; and C1-3 = 10nf):

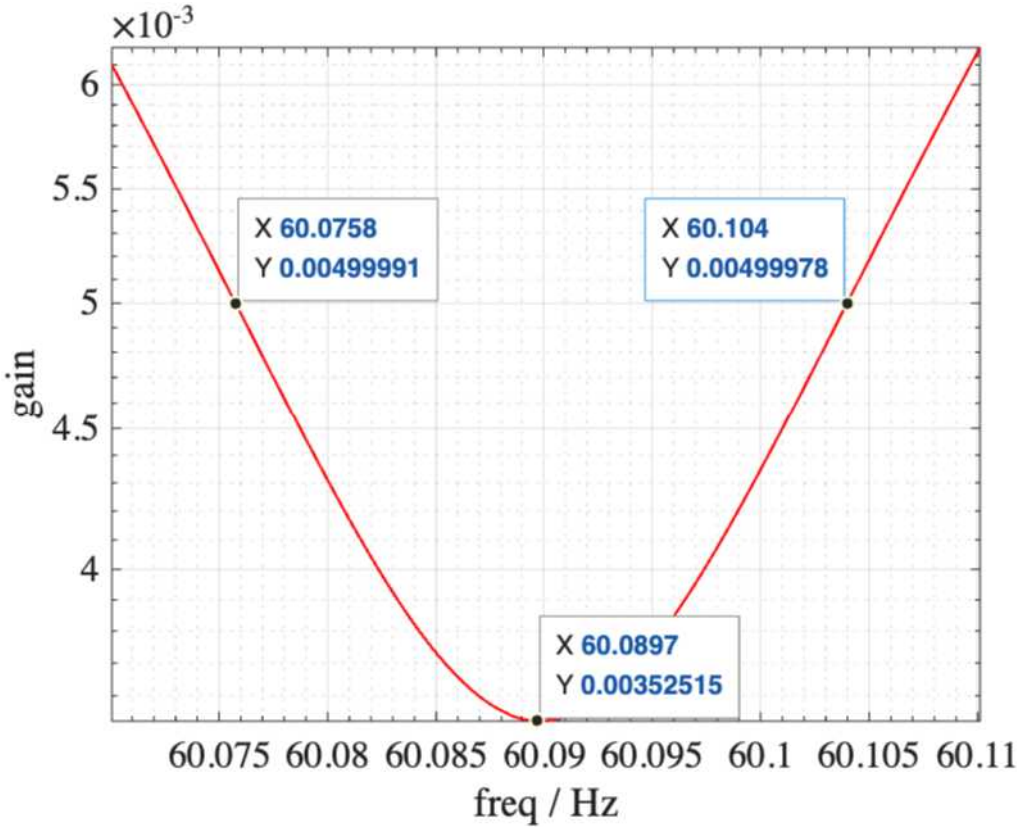



Figure 14. Recomputed gain curve near $f_r$.

```
fprintf('Q = %.0f\n', 60.0897 / (60.104-60.076))
```

```
Q = 2146
```

Are all those figures significant? If we ignore the (implicit) uncertainties in each datum. Parasitic capacitances, leakage resistances in the capacitors, errors or drifts in the parameters, are not included in the model (for good reasons – simplicity). In other words, the uncertainty stems from the fact that *the model is approximate.* How uncertainty in the model propagates into the results can be complicated. Also in practice, making a filter with too high a Q can make it *less* useful, because $f_r$ and gain can drift by significant amounts over important time frames. High Q circuits may have to be placed in temperature-controlled ovens, RF shields, etc. to reduce environmental-induced instabilities.

If I wanted to optimize the circuit over multiple parameter values then a better strategy (i.e., automated, vs. Tool Tips) could be worthwhile. You might first copy the code block and change key variable names to not overwrite prior ones. Changing the number of frequencies to e.g., 999 might be useful to ensure every relevant variable was updated (i.e., by generating an "Arrays have incompatible sizes for this operation" error if not). MATLAB offers many methods to solve problems like this – `spline`, `fminsearch`, neural nets, etc. What *is* the best method? What does "best" mean? The fastest to code and validate? The fastest to execute? The most precise? The most *accurate*? The most confident? The simplest? ... Finding an optimum solution to the wrong metric is a common error, as is "paralysis by analysis" – overthinking a problem.

## Discussion

With minor (and plausible) parameter fudging, the LM317 controller models agree with measurements, suggesting the initial model captured the primary characteristics of the circuit. Models 2 and 3 were academically interesting, they mainly revealed their respective effects are minor. The models disagree with measurements most near the lowest and highest voltages, where the transistors are transitioning to their active region, or the LM317 is approaching saturation against its dropout voltage. Nothing in our models attempts to capture realistic device behaviors around these inflection points, so it's not surprising that errors increase here.

We could attempt to model transistors in their transition regions – there are many sources for the equations (e.g., https://en.wikipedia.org/wiki/Bipolar_junction_transistor#Theory_and_modeling) But my goal as an engineer is usually not to add more significant figures, but to build devices exhibiting robust and predictable behaviors. These models provide additional confidence that the circuits are robust and can be expected to keep working tomorrow and onward.

Another key lesson of this activity is to show you not to fear building simulations in MATLAB starting from first principles. Although specialized circuit simulator software such as SPICE and SIMULINK's electronics toolboxes have powerful capabilities, they can also obfuscate (hide) the dominant and secondary sources of the circuit's characteristics. Here little was hidden, the parameters and interactions are explicit. Likewise, AI's can answer many questions but how they arrived at those answers is generally hidden, and you might not agree with some of its assumptions, data, and reasoning (if you knew what they were).

When complex systems like these circuits are empirically measured, and numerically modeled, and the two sets of results agree, confidence in both the measurements and models increases. If they disagree, then one or both are likely wrong. Progress in science and engineering advances much faster when our non-trivial measurements and models agree. Confronted with a new question, we can turn to the easiest of the two for an answer. Progress in modeling can drive improvements in measurements and visa versa. Without a model (magenta in Fig. 9), would you have realized just from the measurements (black) that a component in a low pass filter had been installed with the wrong value?

What other lessons did you learn?